

Course Name:	Digital Signal and Image Processing	Semester:	VI
Date of Performance:	27 / 03 / 2026	DIV/ Batch No:	C-3
Student Name:	Shreyans Tatiya	Roll No:	16010123325

Title: Compute DFT & IDFT of 2D image using Matlab.

Objective: To learn & understand the Fourier transform operations on discrete time signals.

Expected Outcome of Experiment:

CO	Outcome
CO3	Analyze signals in frequency domain through various image transforms

Books/ Journals/ Websites referred:

1. <http://www.mathworks.com/support/>
2. www.math.mtu.edu/~msgocken/intro/intro.html
3. www.mccormick.northwestern.edu/docs/efirst/matlab.pdf
4. A.Nagoor Kani "Digital Signal Processing", 2nd Edition, TMH Education.

Pre Lab/ Prior Concepts:

Implementation details along with screenshots:

Given a sequence of N samples $f(n)$, indexed by $n = 0..N-1$, the Discrete Fourier Transform (DFT) is defined as $F(k)$, where $k=0..N-1$:

$$F(k) = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} f(n) e^{-j2\pi kn/N}$$

$F(k)$ are often called the 'Fourier Coefficients' or 'Harmonics'.

The sequence $f(n)$ can be calculated from $F(k)$ using the Inverse Discrete Fourier Transform (IDFT):

$$f(n) = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} F(k) e^{+j2\pi nk/N}$$

In general, both $f(n)$ and $F(k)$ are complex.

Annex A shows that the IDFT defined above really is an *inverse* DFT.

Conventionally, the sequences $f(n)$ and $F(k)$ is referred to as 'time domain' data and 'frequency domain' data respectively. Of course there is no reason why the samples in $f(n)$ need be samples of a time dependent signal. For example, they could be spatial image samples (though in such cases a 2 dimensional set would be more common).

Although we have stated that both n and k range over $0..N-1$, the definitions above have a periodicity of N :

$$F(k + N) = F(k) \quad f(n + N) = f(n)$$

So both $f(n)$ and $F(k)$ are defined for all (integral) n and k respectively, but we only need to calculate values in the range $0..N-1$. Any other points can be obtained using the above periodicity property.

For the sake of simplicity, when considering various Fast Fourier Transform (FFT) algorithms, we shall ignore the scaling factors and simply define the FFT and Inverse FFT (IFFT) like this:

$$FFT_N(k, f) = \sum_{n=0}^{N-1} f(n) e^{-j2\pi kn/N} = \sqrt{N} F(k)$$

$$IFFT_N(n, F) = \sum_{k=0}^{N-1} F(k) e^{+j2\pi nk/N} = \sqrt{N} f(n)$$

In fact, we shall only consider the FFT algorithms in detail. The inverse FFT (IFFT) is easily obtained from the FFT.

Implementation steps with screenshots for DFT and IDFT.

Manual-

```
clc;
clear;
close all;

% Read image
img = imread('IMG_6232.jpg');

% Convert to grayscale
if size(img,3) == 3
    img = rgb2gray(img);
end

% Resize image
img = imresize(img, [64 64]);

img = double(img);

[M, N] = size(img);

% Create DFT matrices
W_M = zeros(M, M);
W_N = zeros(N, N);

for k = 0:M-1
    for m = 0:M-1
        W_M(k+1, m+1) = exp(-1i * 2 * pi * k * m / M);
    end
end

for l = 0:N-1
    for n = 0:N-1
        W_N(l+1, n+1) = exp(-1i * 2 * pi * l * n / N);
    end
end

% 2D DFT
F = W_M * img * W_N';
magnitude_spectrum = log(1 + abs(F));

% 2D IDFT
img_rec = (1/(M*N)) * (W_M') * F * (W_N);
img_rec = real(img_rec);

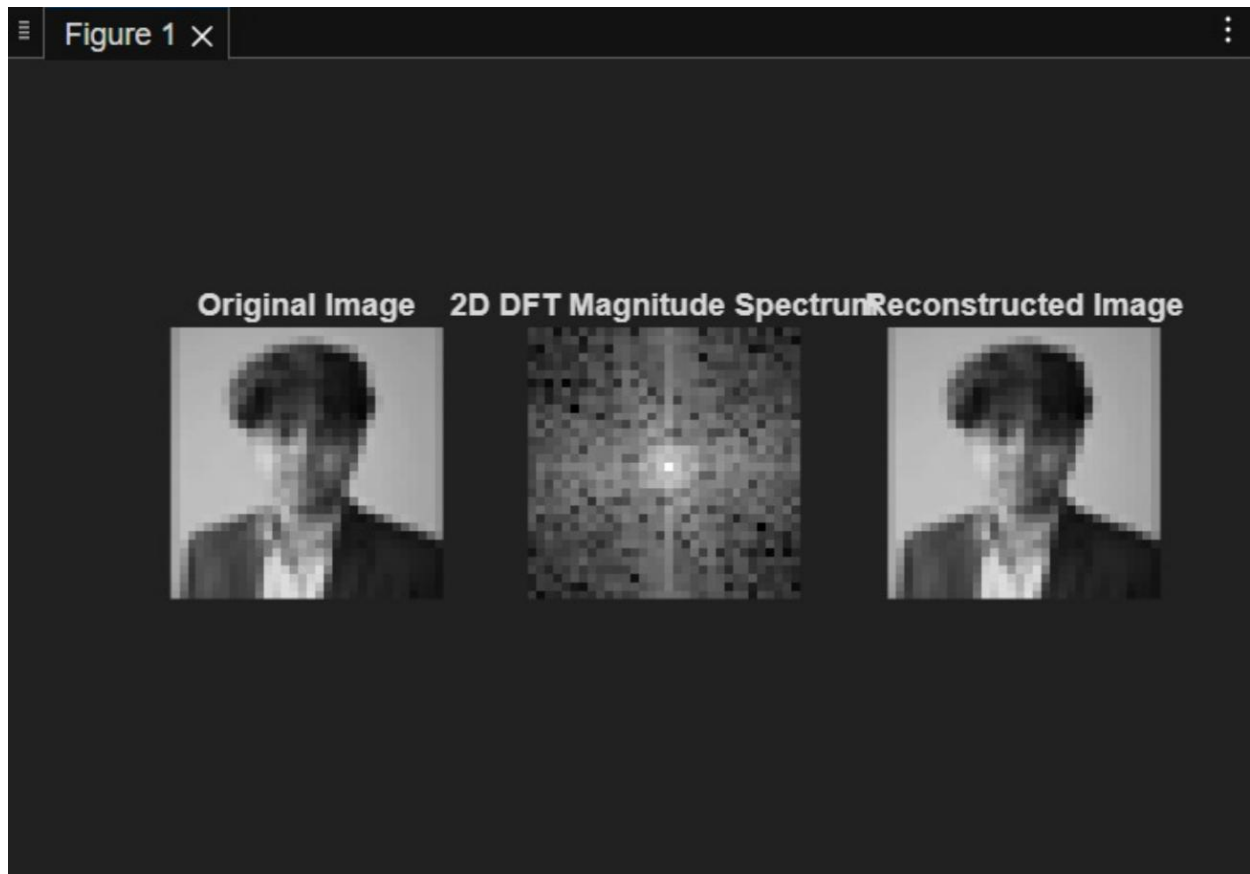
% Display
figure;

subplot(1,3,1);
imshow(uint8(img));
title('Original Image');

subplot(1,3,2);
imshow(magnitude_spectrum, []);
title('2D DFT');
```

```
subplot(1,3,3);  
imshow(uint8(img_rec));  
title('Reconstructed Image');
```

Output-



In-built-

```
clc;
clear;
close all;

% Read image
img = imread('IMG_6232.jpg');

% Convert to grayscale
if size(img,3) == 3
    img = rgb2gray(img);
end

% Resize image
img = imresize(img, [64 64]);

img = double(img);

% 2D DFT using inbuilt function
F = fft2(img);

% Magnitude spectrum
magnitude_spectrum = log(1 + abs(F));

% 2D IDFT using inbuilt function
img_rec = ifft2(F);
img_rec = real(img_rec);

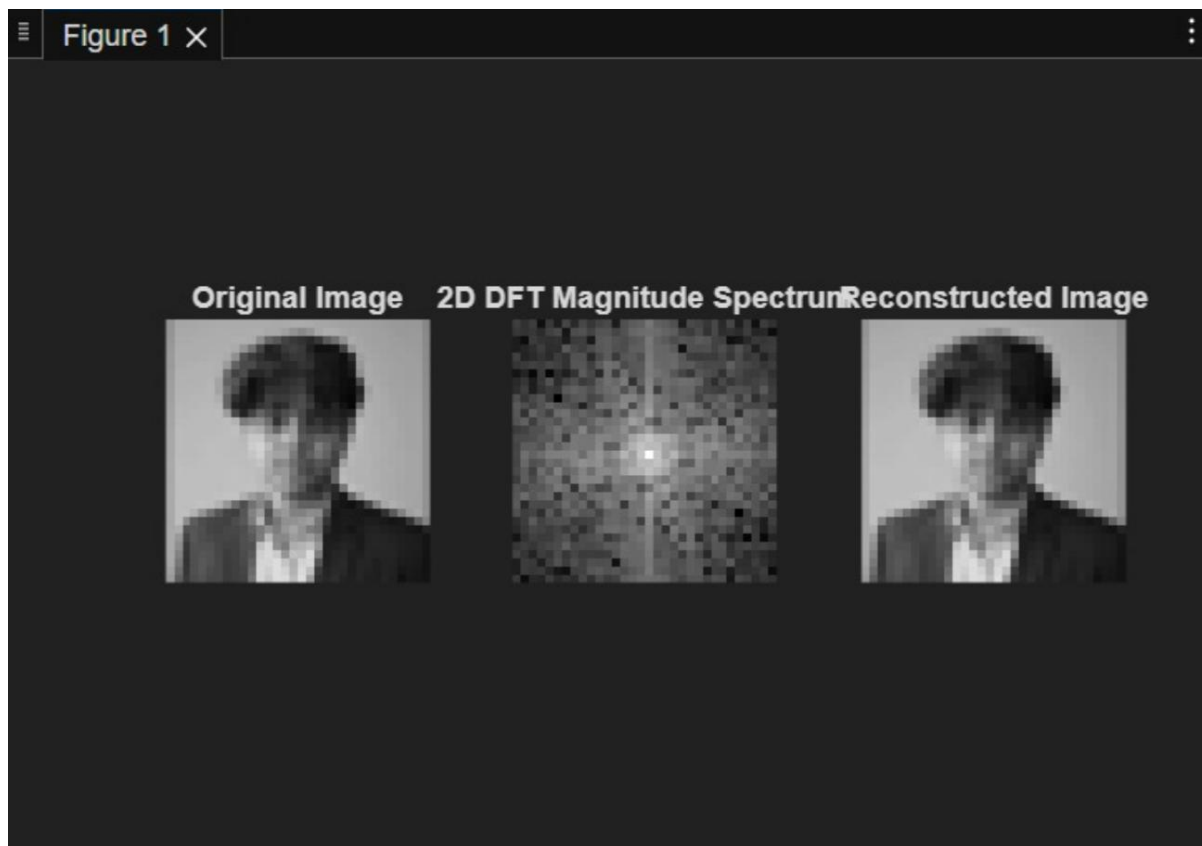
% Display
figure;

subplot(1,3,1);
imshow(uint8(img));
title('Original Image');

subplot(1,3,2);
imshow(magnitude_spectrum, []);
title('2D DFT (Inbuilt)');

subplot(1,3,3);
imshow(uint8(img_rec));
title('Reconstructed Image');
```

Output-



Conclusion:-

In this experiment, we implemented the 2D Discrete Fourier Transform (DFT) using its mathematical matrix formulation and reconstructed the image using the Inverse DFT (IDFT). The results verified that the transformation to frequency domain and back preserves the original image.

Date: _____

Signature of faculty in-charge

Post Lab Descriptive Questions

1. Compare and discuss the computational efficiency of DFT and FFT

DFT

- Uses direct formula (double summation)
- Time complexity: $O(N^2)$ (1D), $O(N^4)$ (2D)
- Very slow for large images

FFT

- Optimized algorithm (divide and conquer)
- Time complexity: $O(N \log N)$
- Much faster and widely used

2. Give the properties of DFT and IDFT.

- **Linearity**
 $af_1 + bf_2 \rightarrow aF_1 + bF_2$
- **Periodicity**
 $F(k+N) = F(k)$
- **Conjugate Symmetry (for real signals)**
 $F(N-k) = F^*(k)$
- **Time Shifting**
Shift in time $\rightarrow$ phase change in frequency
- **Frequency Shifting**
Shift in frequency $\rightarrow$ modulation in time
- **Convolution Property**
Circular convolution in time $\leftrightarrow$ multiplication in frequency

3. Discuss the impact on computation time & efficiency when the number of samples N increases.

- Inverse of DFT
- Restores original signal
- Includes normalization factor $1/N$ or $1/MN$

4. How to compute maximum length N for a circular convolution using DFT and IDFT?

DFT

- Computation increases **very rapidly**
- Time $\propto N^2$ (1D), N^4 (2D)

- Becomes impractical for large N

FFT:

- Time $\propto N \log N$
- Scales efficiently